

JAVA OBJECT-ORIENTED PROGRAMMING

A practical revision guide to classes, objects, the four OOP pillars and class relationships.

1. The Core Idea

CLASS

A blueprint that defines the data (fields) and behaviour (methods) shared by a type of object.

OBJECT

A runtime instance of a class. Each object has its own state but follows the structure defined by its class.

A complete class and object

```
public class Student {
    private String name;
    private int mark;

    public Student(String name, int mark) {
        this.name = name;
        this.mark = mark;
    }

    public String getName() { return name; }
    public int getMark() { return mark; }

    public void study() {
        System.out.println(name + " is studying");
    }
}

Student student = new Student("Lerato", 78);
student.study();
```

Part	Purpose	Example
Field	Stores object state	private int mark;
Constructor	Initialises a new object	new Student("Lerato", 78)
Method	Defines object behaviour	student.study();
Reference	Variable pointing to an object	Student student

REMEMBER

The variable stores a reference to the object; the object itself is created with new.

2. Constructors, this, static and Access

- A constructor has the same name as the class and no return type.
- If you declare no constructor, Java supplies a default no-argument constructor.
- Once you declare any constructor, Java no longer supplies that default constructor.
- `this` refers to the current object; `this(...)` calls another constructor in the same class.

```
public class Account {
    private String owner;
    private double balance;
    private static int accountCount = 0;

    public Account(String owner) {
        this(owner, 0.0);    // constructor chaining
    }

    public Account(String owner, double balance) {
        this.owner = owner;    // field = parameter
        this.balance = balance;
        accountCount++;
    }

    public static int getAccountCount() {
        return accountCount;
    }
}
```

Modifier	Same class	Same package	Subclass elsewhere	Everywhere
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes*	No
public	Yes	Yes	Yes	Yes

* In a subclass outside the package, protected access is through inheritance rules.

Instance vs static

Instance member	Static member
Belongs to each object	Belongs to the class
Access with an object reference	Prefer access with <code>ClassName.member</code>
Can directly use instance and static members	Cannot directly use instance members
Example: <code>balance</code>	Example: <code>accountCount</code>

DESIGN TIP

Keep fields private. Expose only the operations the rest of the program genuinely needs.

3. Encapsulation and Inheritance

Encapsulation

Bundle data and behaviour together, then protect the object's valid state. Encapsulation is more than adding getters and setters: methods should enforce the class rules.

```
public class BankAccount {
    private double balance;

    public double getBalance() { return balance; }

    public void deposit(double amount) {
        if (amount <= 0)
            throw new IllegalArgumentException("Amount must be positive");
        balance += amount;
    }
}
```

Inheritance

A subclass receives accessible members from a superclass. Use inheritance only for a genuine is-a relationship.

```
public class Person {
    protected String name;

    public Person(String name) { this.name = name; }
    public void introduce() { System.out.println(name); }
}

public class Student extends Person {
    private String studentNumber;

    public Student(String name, String number) {
        super(name); // superclass constructor first
        this.studentNumber = number;
    }

    @Override
    public void introduce() {
        System.out.println(name + " - " + studentNumber);
    }
}
```

Keyword	Meaning
extends	Creates a class inheritance relationship
super(...)	Calls a superclass constructor; must be the first constructor statement
super.member	Accesses an accessible superclass member
@Override	Asks the compiler to verify that a method is correctly overridden
final class	Cannot be extended
final method	Cannot be overridden

RULE Java supports single class inheritance. A class may extend one class but implement multiple interfaces.

4. Polymorphism and Abstraction

Runtime polymorphism (overriding)

A superclass or interface reference can point to different subclass objects. Java chooses the overridden instance method using the object's actual runtime type.

```
Person person = new Student("Lerato", "ST100");
person.introduce(); // calls Student.introduce()

Person[] people = {
    new Student("Lerato", "ST100"),
    new Lecturer("Simon", "Programming")
};
for (Person p : people) p.introduce();
```

Compile-time polymorphism (overloading)

```
void print(int value) { ... }
void print(String value) { ... }
void print(String value, int copies) { ... }
```

Overloading	Overriding
Same method name, different parameter list	Subclass replaces inherited method behaviour
Usually in the same class	Requires inheritance
Chosen at compile time	Chosen at runtime for instance methods
Return type alone cannot distinguish methods	Same/covariant return type and compatible access

Abstract class

```
public abstract class Shape {
    public abstract double area();
    public void describe() { System.out.println("A shape"); }
}
```

Interface

```
public interface Drawable {
    void draw(); // implicitly public abstract
}

public class Circle extends Shape implements Drawable {
    private double radius;
    public double area() { return Math.PI * radius * radius; }
    public void draw() { System.out.println("Drawing circle"); }
}
```

PROGRAM TO A TYPE

Use a superclass or interface reference when calling code only needs the shared contract, not subclass-specific details.

5. Object Relationships

Prefer composition when one object uses or owns another object and there is no true is-a relationship.

Relationship	Meaning	Example
Inheritance	is-a	Student is a Person
Association	uses/knows-a; independent lifetimes	Lecturer teaches Student
Aggregation	weak whole-part; part can exist alone	Team has Players
Composition	strong whole-part; owner manages lifetime	House has Rooms
Dependency	temporary use, often parameter/local variable	Report uses Printer

Composition example

```
public class Engine {
    public void start() { System.out.println("Engine started"); }
}

public class Car {
    private final Engine engine;

    public Car() {
        engine = new Engine();    // Car owns its Engine
    }

    public void start() {
        engine.start();           // delegation
    }
}
```

Abstract class vs interface

Abstract class	Interface
Can contain instance fields and constructors	No instance fields or constructors
Can contain abstract and concrete methods	Defines a contract; may also have default/static/private methods
A class can extend only one	A class can implement multiple
Use for closely related classes sharing state/implementation	Use for a capability or role across unrelated classes

CHOOSE WELL

Prefer composition for flexible code reuse. Choose inheritance when substituting the child wherever the parent is expected makes logical sense.

6. Quick Reference and Common Traps

Concept	Java syntax / reminder
Create object	Type ref = new Type(arguments);
Inheritance	class Child extends Parent
Interface	class Type implements Contract
Override	@Override above the subclass method
Call current constructor	this(arguments); first statement
Call parent constructor	super(arguments); first statement
Type check	obj instanceof Type
Safe modern cast	if (obj instanceof Student s) { ... }
Prevent reassignment	final Type value
Object comparison	Use equals(); use == for reference identity

Common mistakes

- Giving every field a public setter, allowing invalid object state.
- Using == instead of equals() to compare object content.
- Overloading when overriding was intended because the parameter list differs.
- Forgetting @Override, which would have exposed the signature mistake.
- Downcasting without checking the object's actual type.
- Calling an overridable method from a constructor before the subclass is fully initialised.
- Using inheritance only to reuse code, despite there being no logical is-a relationship.

Mini example: polymorphic design

```
interface Payable {
    double calculatePay();
}

class Tutor implements Payable {
    private int hours;
    private double hourlyRate;

    public double calculatePay() {
        return hours * hourlyRate;
    }
}

static void printPayment(Payable worker) {
    System.out.println(worker.calculatePay());
}
```

REVISION TEST

Can you explain why Payable worker improves flexibility, and identify encapsulation, abstraction and polymorphism in the example?

LEARN • PRACTISE • GROW

PerfectIT Programming Tutoring